

Recuperação de Banco de Dados

Técnicas de recuperação baseadas em Log

- Esta é a abordagem mais comum e utiliza um arquivo de **Log** (ou *Journal*) para registrar sequencialmente todas as atualizações feitas pelas transações. O log é consultado para realizar operações **UNDO** e **REDO**
 - tipos de log
 - log de UNDO – mantém apenas o valor antigo do dado (before image)
 - log de REDO – mantém apenas o valor atualizado do dado (after image)
 - log de UNDO/REDO (mais comum) – mantém os valores antigo e atualizado do dado

– início de transação: `<start Tx>`

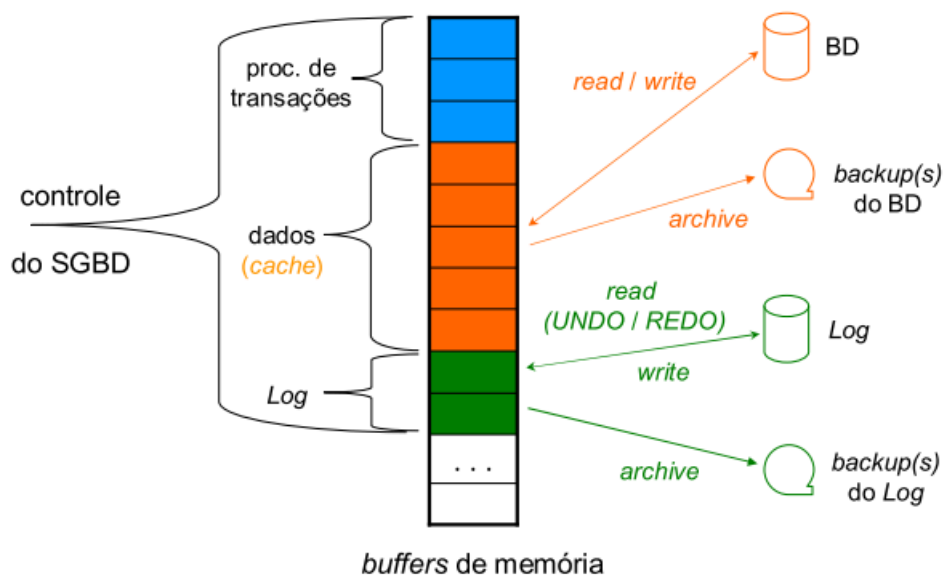
– *commit* de transação: `<commit Tx>`

– atualização: `<write Tx, X, beforeImage, afterImage>`

não é necessário em log REDO

não é necessário em log UNDO

Gerenciamento do Buffer



- Princípio Write-Ahead-Log (WAL):** Antes de um bloco de dados ser modificado no disco, o registro de log correspondente a essa modificação **deve** ser gravado no disco primeiro.

Gerenciamento de Buffer e sua influência na recuperação

- **STEAL**: Permite que um bloco modificado na cache por uma transação ainda não concluída seja gravado no disco. Isso requer a necessidade de **UNDO** em caso de falha.
- **NOT-STEAL**: Proíbe que um bloco modificado seja gravado no disco antes do commit da transação. Simplifica a recuperação, pois elimina a necessidade de **UNDO**.
- **FORCE**: Força a gravação no disco de todos os blocos modificados por uma transação assim que ela realiza o commit. Garante a durabilidade, mas pode ser ineficiente.
- **NOT-FORCE**: Não força a gravação imediata dos blocos após o commit. Melhora o desempenho, mas exige a necessidade de **REDO** em caso de falha.

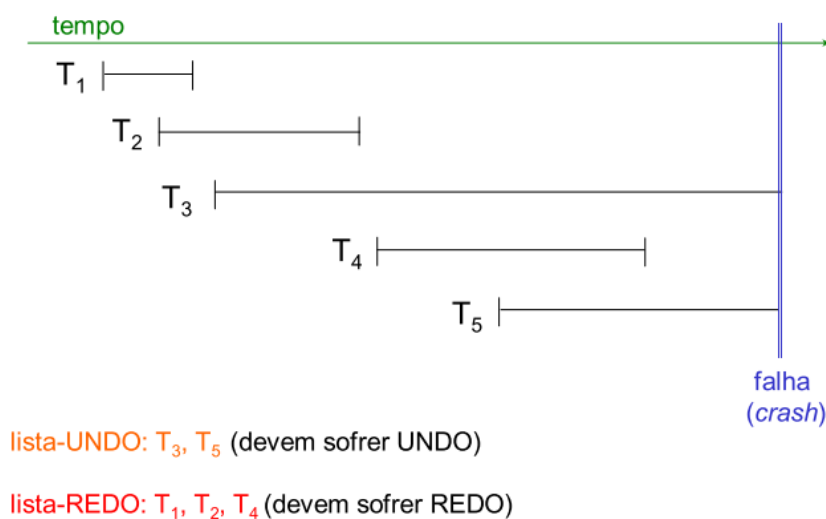
Algoritmos de recuperação baseados em Log

Modificação Imediata do BD (STEAL / NOT-FORCE)

- As atualizações podem ser gravadas no disco antes do commit da transação.

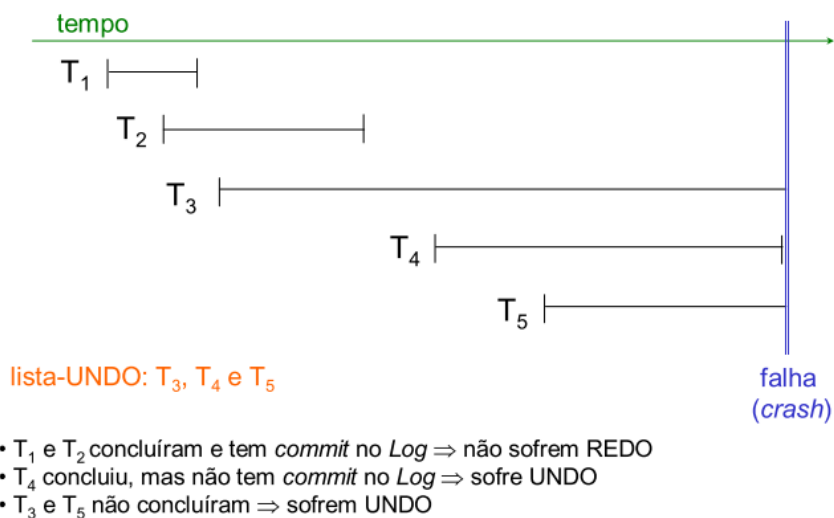
Técnica UNDO/REDO: É a mais comum.

- **Procedimento**: Após uma falha, transações que iniciaram mas não possuem um **<commit>** no log devem sofrer **UNDO**. Transações que possuem um **<commit>** no log devem sofrer **REDO** para garantir que suas alterações estejam no disco.
- O processo geralmente envolve uma varredura para trás (*backward*) do log para **UNDO** e uma para frente (*forward*) para **REDO**.



- **Técnica UNDO/NO-REDO**: Usa a política **FORCE**.
 - **Procedimento**: Como as alterações são forçadas no disco no momento do commit, não há necessidade de **REDO**. Apenas as transações não concluídas precisam sofrer **UNDO**.

Técnica UNDO/NO-REDO - Exemplo



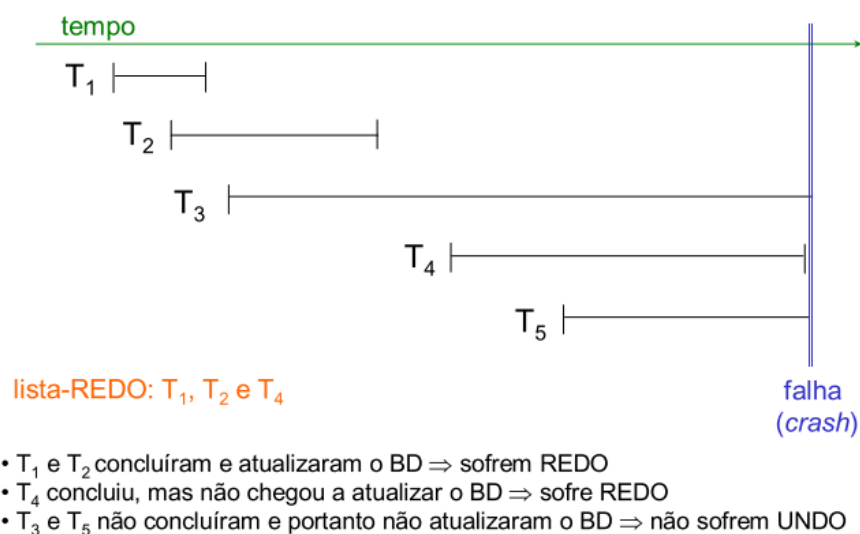
Modificação Postergada do BD (NOT-STEAL / NOT-FORCE)

As atualizações de uma transação só podem ir para o disco após o seu commit.

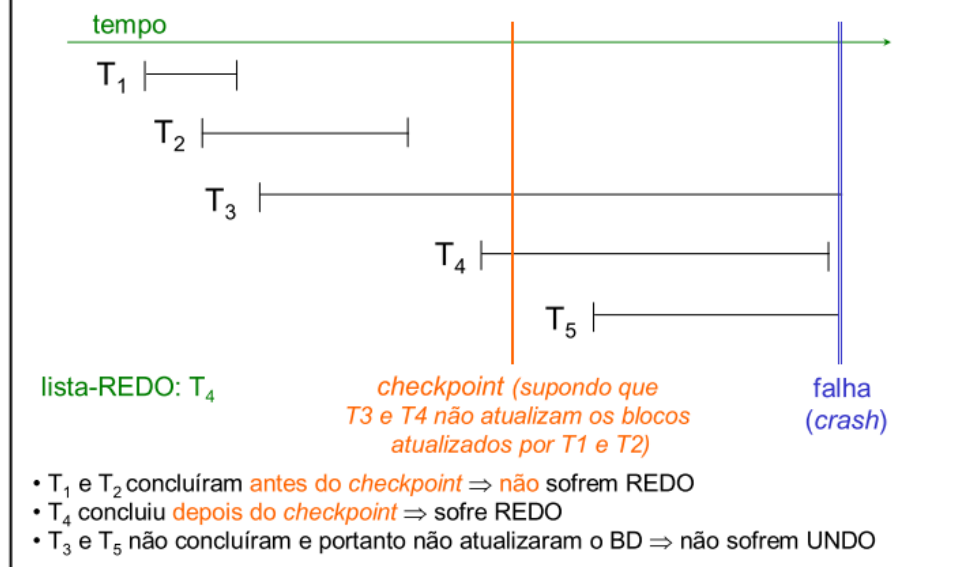
- **Técnica NO-UNDO/REDO:**

- **Procedimento:** Como nenhuma alteração de transações inacabadas vai para o disco, não há necessidade de **UNDO**. Apenas as transações com **<commit>** no log precisam sofrer **REDO** para garantir que suas atualizações sejam gravadas.

Técnica NO-UNDO/REDO - Exemplo



NO-UNDO/REDO c/ Checkpoint

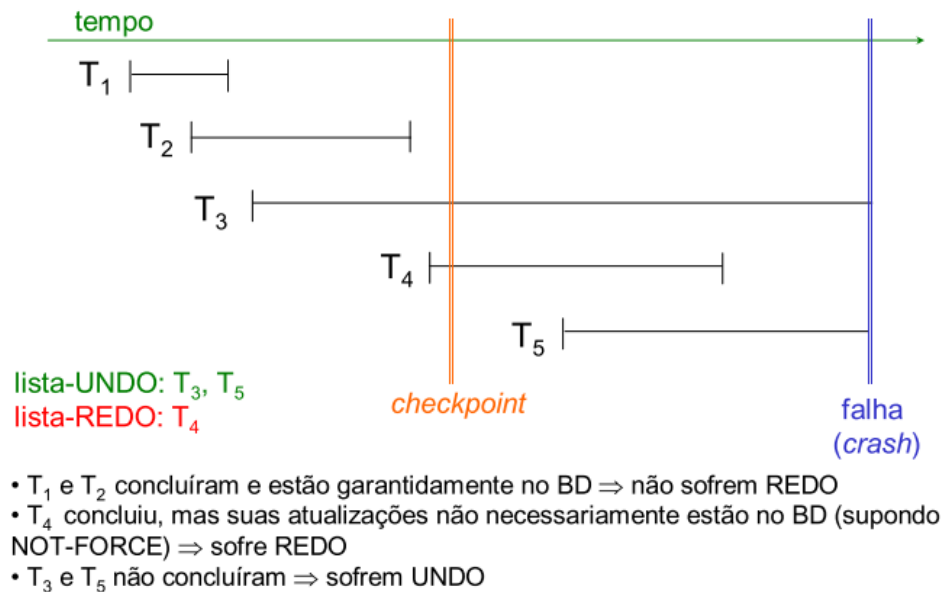


Checkpoints

Para otimizar a recuperação em sistemas com logs muito grandes, utiliza-se a técnica de **Checkpoint**.

- **O que é:** Um ponto no tempo em que o SGBD garante que todas as atualizações de transações já concluídas foram gravadas no disco. Um registro `<checkpoint T1, T2...>` é inserido no log, listando as transações que estavam ativas naquele momento.
- **Vantagem:** Durante a recuperação, o SGBD só precisa analisar o log a partir do último checkpoint. Transações que fizeram commit *antes* do checkpoint não precisam de **REDO**, pois suas alterações já estão garantidamente no disco.

Técnica UNDO/REDO c/ Checkpoint



Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – **Passo 1**

Percorre-se o *Log backward* até alcançar um registro *Checkpoint*

- se achou <commit T_x>, insere T_x na **lista-REDO**
- se achou <start T_x> e T_x não está na lista-REDO, insere T_x na **lista-UNDO**

Exemplo de *Recovery* com *Checkpoint*

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Lista UNDO:

Lista REDO:

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

(não necessariamente todos os
writes foram persistidos no BD)

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Após a execução do Passo 1

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Lista UNDO: **T6, T5**

Lista REDO: **T7, T3**

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 2

Analisa-se cada transação T_x no registro *checkpoint*

- se T_x não estiver na lista-REDO, insere T_x na lista-UNDO

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3(T4)>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 2

Lista UNDO: T6, T5, **T4**

Lista REDO: T7, T3

→ neste ponto já se conhece todas as transações que devem sofrer UNDO ou REDO !

Situação no BD
após a falha:

A = 25
B = 20
C = 65
D = 39
E = 17
F = 2

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 3

Percorre-se de novo o *Log backward*, até que todas as transações em lista-UNDO tenham sofrido UNDO

- marca-se na lista-REDO as transações T_x cujos registros $\langle \text{start } T_x \rangle$ estão sendo encontrados nessa varredura

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 3

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~

Lista REDO: T7, T3

Situação no BD
após a falha:

A = 25
B = 20
C = ~~65~~ 55
D = ~~39~~ 31
E = ~~17~~ 28
F = ~~2~~ 1

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 4

Caso existam transações não marcadas na lista-REDO ao final da varredura *backward*

- continua-se a varredura *backward* até que todas as transações na lista-REDO tenham sido marcadas

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 4

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~

Lista REDO: T7, T3
✓ ✓

Situação no BD
após a falha:

A = 25
B = 20
C = ~~65~~ 55
D = ~~39~~ 31
E = ~~17~~ 28
F = ~~2~~ 1

Desejável:

A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Log

Técnica UNDO/REDO c/ Checkpoint

- Procedimento de *Recovery* – Passo 5

Percorre-se o *Log forward* do ponto de parada, realizado REDO das transações na lista-REDO

```
<start T1>
<write T1,A,5,10>
<start T2>
<write T2,C,30,45>
<write T2,E,7,17>
<commit T2>
<write T1,C,45,55>
<start T3>
<write T3,B,15,20>
<commit T1>
<start T4>
<write T4,C,55,65>
<checkpoint T3,T4>
<start T5>
<write T5,D,31,39>
<start T6>
<write T3,A,5,25>
<write T6,F,1,2>
<write T3,E,17,28>
<commit T3>
<write T6,A,25,32>
<start T7>
<write T7,B,20,30>
<commit T7>
<write T4,E,28,34>
Crash!
```

Após a execução do Passo 5

Lista UNDO: ~~T6~~, ~~T5~~, ~~T4~~
Lista REDO: ~~T7~~, ~~T3~~

Situação no BD após a falha:

A = 25	✓
B = 20	✓ 20 30
C = 65	55
D = 39	31
E = 17	28 ✓
F = 2	1

Desejável:
A = 25
B = 30
C = 55
D = 31
E = 28
F = 1

Vantagem da técnica:
Não é necessário varrer todo o Log para realizar o *Recovery*!

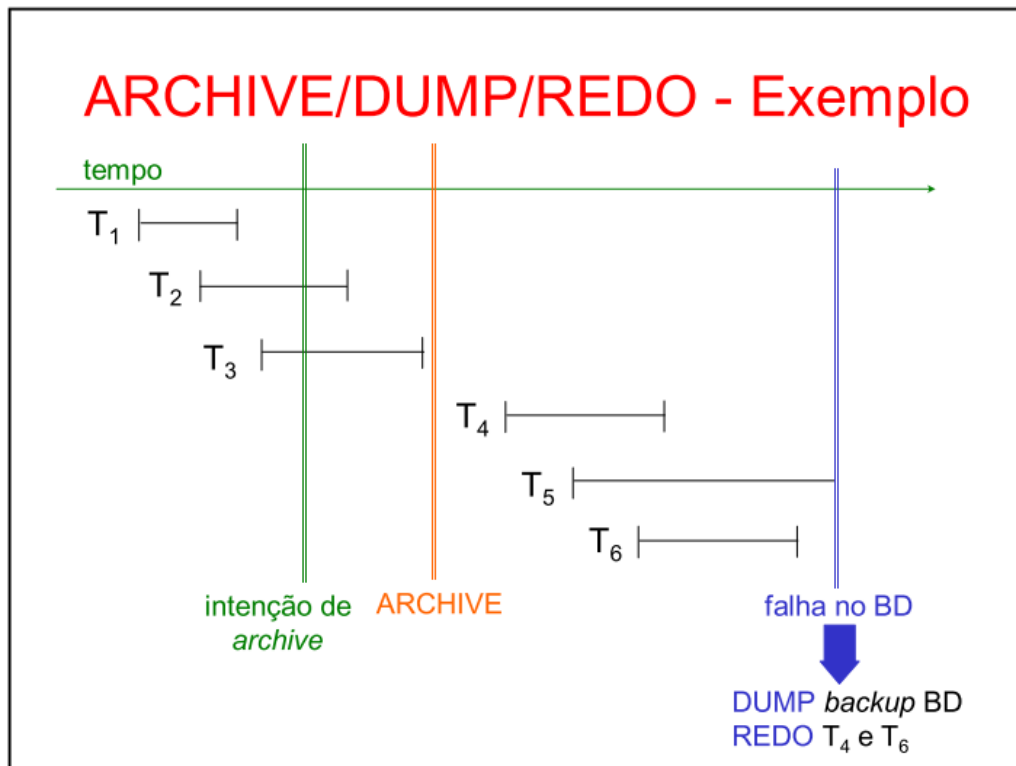
Recuperação de falha de meio de armazenamento

Técnica **ARCHIVE/DUMP/REDO**: Usada para falhas de disco.

- Procedimento:

1. Restaura-se o banco de dados a partir do último backup (DUMP).

2. Realiza-se uma varredura para frente (*forward*) no log (que também deve ter um backup) para refazer (**REDO**) todas as transações que foram concluídas desde o último backup.



Técnica de Shadow Pages (NO_UNDO/NO-REDO)

Uma abordagem alternativa que não utiliza log.

- **Como funciona:** Mantém duas tabelas de páginas de disco: a Corrente (TPC) e a Sombra (TPS). A TPS é uma cópia da TPC no início da transação. Quando a transação atualiza uma página, uma nova cópia da página é criada em um local livre do disco, e a TPC é atualizada para apontar para essa nova página. A TPS permanece inalterada.
- **Recuperação:** Se a transação falhar antes do commit, a TPC modificada é simplesmente descartada, e a TPS (que aponta para o estado original dos dados) torna-se a nova TPC. Não são necessários **UNDO** ou **REDO**.
- **Desvantagens:** É complexa para ambientes multiusuário, pode causar fragmentação no disco (não mantém dados clusterizados) e requer um processo de "coleta de lixo" para gerenciar páginas obsoletas.